

Relazione Progetto MapReduce - libmr

Framework per analisi di file di testo

Maggio 2026

1 Introduzione

Il progetto consiste nella realizzazione di un framework denominato `libmr`, scritto in linguaggio C, che implementa il paradigma MapReduce per l'analisi di file testuali su un singolo calcolatore. Il framework gestisce la creazione di processi, la comunicazione tramite pipe e la sincronizzazione di thread, permettendo all'utente di concentrarsi esclusivamente sulla logica di *mapping* e *reducing*.

Allo stato attuale dello sviluppo, il framework implementa l'interfaccia pubblica, comprensiva della gestione della configurazione, e il sistema di logging sincronizzato. Le componenti relative alla pipeline di processi e all'esecuzione multithread sono in fase di progettazione e implementazione.

2 Architettura Generale

L'architettura prevista per il framework si basa su una pipeline di tre processi:

- **Processo Principale:** Si occupa della lettura dei file di input e dell'invio delle righe al mapper. Raccoglie inoltre i risultati finali dal reducer.
- **Processo Mapper:** Riceve le righe, le elabora tramite thread C11 e invia coppie `<token, valore>` al reducer.
- **Processo Reducer:** Raggruppa i valori per token, esegue la riduzione multithread e produce l'output finale.

Attualmente, il framework fornisce le strutture dati di base (`mr_t`) e le funzioni per la loro gestione, preparando il terreno per l'attivazione della pipeline di processi.

3 Interfaccia Pubblica

L'interfaccia del framework è organizzata per separare la definizione delle API pubbliche dalla configurazione dei parametri interni.

3.1 `mr.h`

L'header principale `include/mr.h` implementa fedelmente l'interfaccia pubblica richiesta nel testo della specifica del progetto, garantendo la compatibilità con i programmi di test esterni e fornendo tutti i tipi, le strutture e i prototipi di funzione necessari all'utilizzo della libreria.

3.2 config.h

L'header `include/config.h` definisce le costanti e i valori di default utilizzati dal framework. Grazie all'uso di direttive `#ifndef`, questi valori possono essere sovrascritti con flag di compilazione (es. `-DMR_LOG_LEVEL=MR_LOG_LEVEL_WARNING`).

Di seguito la descrizione delle principali macro:

- **MR_DEFAULT_MAPPER_THREADS**: Numero predefinito di thread worker avviati nel processo mapper (valore: 4).
- **MR_DEFAULT_REDUCER_THREADS**: Numero predefinito di thread worker avviati nel processo reducer (valore: 4).
- **MR_DEFAULT_QUEUE_SIZE**: Capacità massima predefinita per le code produttore-consumatore interne (valore: 100).
- **MR_DEFAULT_LOG_FILE**: Nome predefinito del file su cui vengono scritti i log ("`mr.log`").
- **MR_LOG_SEM_NAME**: Nome del semaforo POSIX utilizzato per sincronizzare l'accesso al file di log (valore: "`/mr_log_sem`").
- **MR_LOG_LEVEL**: Definisce la verbosità del sistema di logging. Di default è impostato su `MR_LOG_LEVEL_INFO`, o `MR_LOG_LEVEL_DEBUG` se compilato in modalità debug.

4 Sistema di Logging

Il sistema di logging è una componente fondamentale già completamente operativa. Le sue caratteristiche principali includono:

- **Sincronizzazione**: Utilizzo di semafori POSIX (`sem_t`) per garantire l'atomicità delle scritture da processi e thread diversi.
- **Formato**: Ogni riga di log segue rigorosamente il formato:

```
<timestamp> <pname>[<pid>] <tname> <level>: <message>
```

- **Persistenza**: I log vengono scritti in modalità *append* sul file configurato.

5 Descrizione dei Test Realizzati

Sono stati implementati test unitari utilizzando il framework **Unity** per verificare la robustezza dell'interfaccia pubblica e della gestione degli attributi. I test coprono:

- Corretta inizializzazione con i valori predefiniti nel sistema.
- Rifiuto di parametri non validi e impostazione corretta di `errno`.
- Gestione sicura della memoria nelle operazioni di creazione e distruzione.

6 Conclusioni

Allo stato attuale, `libmr` fornisce una solida base infrastrutturale e un'interfaccia utente coerente. Le prossime fasi di sviluppo si concentreranno sull'attivazione della pipeline di processi tramite `fork()` e `pipe()` e sull'orchestrazione dei thread worker all'interno di mapper e reducer.